SmartPrice

Hyperlocal Quick-Commerce Price Comparison Engine

High-Level Design (HLD), Detailed-Level Design (DLD) & Pitch Deck for Android (Kotlin)

Target Platforms:	Blinkit & Zepto (Expandable to Instamart, BigBasket, Amazon Fresh)
Mobile Tech Stack:	Native Android (Kotlin), Jetpack Compose, Coroutines, Flow, Retrofit
Backend Architecture:	Python (FastAPI) / Node.js Microservice with Location-Aware Scraper Pipeline
Document Version:	1.0 (Proof of Concept - Production Blueprint)
Author / Project:	SmartPrice Engineering & Product Architecture Team
API Cost / Charges:	■0 (100% Free Public Web Ingestion for POC)

Executive Summary:

Consumers in urban India frequently toggle between multiple quick-commerce apps (Zepto, Blinkit, Instamart) to check product availability, delivery times, and price disparities. **SmartPrice** is a unified mobile aggregator that allows users to search for any item, resolves the user's nearest dark stores via GPS coordinates, compares real-time prices and delivery ETAs side-by-side, highlights the cheapest option, and enables seamless one-tap purchase via Android deep linking.

1. Problem Statement & Market Opportunity

The Consumer Dilemma:

- **Price Arbitrage:** Identical FMCG items (e.g., Amul Milk, Tide Detergent, Coca-Cola) often carry 5% to 25% price differences across Blinkit and Zepto due to algorithmic dynamic pricing.
- **Hyperlocal Dark Store Variance:** Inventory and pricing are not nationwide; they are strictly tied to the user's physical GPS location (within 2-3 km radii).
- **Friction of App Switching:** Users open 3 separate apps, search the same item 3 times, compare manual delivery fees, and lose time.

Core Objectives of the POC:

1. **Single-Query Search:** User types an item name once and retrieves combined results from both Blinkit and Zepto in < 1.5 seconds.
2. **Live GPS Location Ingestion:** Pass latitude and longitude dynamically to query the exact dark stores serving the user.
3. **Cheapest Indicator:** Clearly tag the lowest-price platform with exact MRP, discounted price, and estimated delivery minutes.
4. **Direct Deep Linking:** Tap 'Buy' to immediately launch the respective store app on Android or fall back to their mobile site.

2. High-Level Design (HLD)

2.1 End-to-End System Architecture:

The architecture is designed as a decoupled 3-tier system: Mobile Client (Android/Kotlin), Unified Aggregator API Gateway (Python/FastAPI), and Hyperlocal Fetching Workers (Blinkit & Zepto Scraper Nodes).

Layer	Component	Key Responsibilities
Presentation	Android App (Kotlin)	Jetpack Compose UI, GPS acquisition (FusedLocation), Search input, StateFlow rendering, Deep link intent launching.
API Gateway	Aggregator Microservice	Validates search queries & GPS coords, coordinates concurrent upstream async calls, aggregates and normalizes raw JSON.
Ingestion Layer	Platform Adapters	**Blinkit Adapter:** Queries Blinkit web API with Lat/Lng. **Zepto Adapter:** Queries Zepto web API with Lat/Lng.
Matching Engine	Normalization Engine	Fuzzy matching on Title + Brand + Pack size (e.g., '500 ml' vs '0.5 L') to combine identical items into a single comparison card.
Caching (Optional)	Redis In-Memory Cache	Caches product searches per Lat/Lng grid for 15 minutes to reduce upstream hits and achieve sub-100ms response times.

2.2 Architectural Data Flow:

1. User opens SmartPrice app $\rightarrow$ App requests GPS permission $\rightarrow$ Acquires `(Lat: 12.9716, Lng: 77.5946)`.
2. User types *'Amul Butter'* $\rightarrow$ Android ViewModel executes Retrofit call to `/api/v1/compare?query=amul+butter■=12.9716&lng;=77.5946`.
3. Backend microservice spawns 2 concurrent asynchronous HTTP workers (Blinkit Worker + Zepto Worker).
4. Upstream responses parsed $\rightarrow$ Filter out of stock items $\rightarrow$ Fuzzy Matcher groups products $\rightarrow$ Calculates cheapest store.
5. Android client receives clean unified JSON payload and displays cards with green 'CHEAPEST' badges.
6. User clicks 'Buy on Zepto' $\rightarrow$ Android Intent launches Zepto application or web fallback.

3. Detailed-Level Design (DLD / LLD)

3.1 Android (Kotlin) Client Architecture:

The Android client adheres strictly to **Google's Recommended Modern Android Architecture (MVVM + Clean Architecture)** using declarative **Jetpack Compose** for UI and **Kotlin Coroutines/StateFlow** for reactive state management.

- **UI Layer:** Jetpack Compose (`ComparisonScreen`, `ProductCard`, `StoreBadge`, `SearchBarComponent`).
- **ViewModel Layer:** `ComparisonViewModel` exposes immutable `StateFlow` (`Idle`, `Loading`, `Success`, `Error`).
- **Domain / Repository Layer:** `ComparisonRepositoryImpl` coordinates location provider and API calls.
- **Data Layer:** Retrofit 2 + OkHttp 4 client configured with timeouts and logging interceptors.
- **Location Services:** Google Play Services `FusedLocationProviderClient` with `PRIORITY_BALANCED_POWER_ACCURACY`.

3.2 Backend API Specifications:

Endpoint:	GET /api/v1/compare
Query Parameters:	query (String, required): Search term (e.g., 'milk') lat (Float, required): User latitude (e.g., 12.9716) lng (Float, required): User longitude (e.g., 77.5946)
Response Status:	200 OK on success | 400 Bad Request on missing coords | 502 Bad Gateway if upstream down

Unified Response Schema (JSON):


```
{
  "status": "success",
  "query": "Amul Butter",
  "location": { "lat": 12.9716, "lng": 77.5946 },
  "totalResults": 1,
  "products": [
    {
      "id": "prod_amul_butter_100g",
      "title": "Amul Pasteurised Butter",
      "packSize": "100 g",
      "imageUrl": "https://cdn.grofers.com/app/images/products/amul_butter.jpg",
      "cheapestStore": "Zepto",
      "maxSavings": 2.0,
      "offers": [
        {
          "store": "Zepto",
          "price": 58.0,
          "mrp": 60.0,
          "inStock": true,
          "eta": "10 mins",
          "deepLink": "https://www.zeptonow.com/pn/amul-pasteurised-butter/pvid/102",
          "packageName": "com.zeptoconsumerapp"
        },
        {
          "store": "Blinkit",
          "price": 60.0,
          "mrp": 60.0,
          "inStock": true,
          "eta": "14 mins",
          "deepLink": "https://blinkit.com/prn/amul-butter/prid/204",
          "packageName": "com.grofers.customerapp"
        }
      ]
    }
  ]
}
```


3.3 Cross-Platform Product Matching Algorithm:

Because Zepto and Blinkit use different catalog naming conventions, matching is done in 3 stages:

1. **Standardized Unit Tokenizer:** Convert '500ml', '500 ml', '0.5L' into canonical '500_ml'.
2. **Brand & Stopword Extraction:** Extract verified brands (e.g. *Amul, Britannia, Nestle, Coca-Cola*) and remove filler words (e.g. *'Fresh', 'Delicious', 'Pouch'*).
3. **Token Set Ratio Matching:** Calculate string similarity score $> 85\%$. Items meeting threshold with identical normalized pack sizes are combined into one comparison card.

4. Project Presentation & Pitch Deck (10 Slides)

The following 10 slides provide a structured presentation suitable for product showcases, stakeholders, and engineering reviews:

Slide 1: Title & Executive Summary — SmartPrice: Next-Gen Hyperlocal Price Comparison Engine

- **Vision:** Eliminate quick-commerce overspending by empowering shoppers with instant price comparison across Blinkit, Zepto, and Instamart.
- **Value Proposition:** Save consumers 10-20% on monthly grocery bills while reducing app-switching friction to zero.
- **Target Audience:** Urban shoppers, bargain hunters, daily FMCG buyers in Tier 1 & Tier 2 cities.

Slide 2: The Core Problem — Quick Commerce Fragmentation & Dynamic Price Discrepancies

- **Price Variance:** Dark stores dynamically adjust prices based on local demand; identical products carry significant price gaps across apps.
- **Hyperlocal Barriers:** No centralized platform allows users to compare prices for their exact GPS block.
- **Wasted Time & Money:** Users spend 5-10 minutes checking multiple apps or settle for higher prices out of convenience.

Slide 3: The SmartPrice Solution — Real-Time Side-by-Side Aggregation & 1-Tap Checkout

- **Single Search:** Type once, compare everywhere in under 1.5 seconds.
- **Hyperlocal Dark Store Awareness:** Automatically routes queries to dark stores serving user's live latitude/longitude.
- **Cheapest Badge:** Instant visual highlighting of lowest cost, discounts, and delivery ETAs.
- **Seamless Routing:** Deep links open the store app directly to the product checkout.

Slide 4: Key Platform Features — POC Capabilities vs Full Scale Product

- **POC Scope:** Blinkit + Zepto comparison for FMCG, Dairy, Beverages, and Essentials.
- **Smart Sorting:** Filter by Lowest Price, Fastest Delivery (ETA), or Highest Discount.
- **Basket Optimizer (Phase 2):** Total grocery list optimization taking delivery fees into account.
- **Price Drop Alerts (Phase 3):** Push notifications when wishlisted products hit all-time low prices.

Slide 5: High-Level Architecture (HLD) — Modern, Resilient, and Scalable 3-Tier Design

- **Client Tier:** Native Android Kotlin app with Jetpack Compose & GPS location resolution.
- **API Gateway:** High-performance FastAPI backend coordinating parallel async worker tasks.
- **Ingestion Engine:** Web endpoint scrapers with User-Agent rotation and location header injection.
- **Cache & Resilience:** Redis caching layer protecting against upstream throttling and rate limits.

Slide 6: Android Client Design (DLD) — Modern Android Architecture (MVVM + Jetpack Compose)

- **UI / UX:** Declarative, silky-smooth Compose UI with responsive comparison cards & badges.
- **Reactive State:** Kotlin StateFlow & Coroutines ensuring zero UI freezes during searches.
- **Networking:** Robust Retrofit 2 client with automatic retry policies and error handling.
- **Intent Engine:** Launches native Android app (com.grofers.customerapp / com.zeptoconsumerapp) with browser fallback.

Slide 7: Ingestion & Fuzzy Matching Engine — Overcoming Unofficial APIs & Catalog Differences

- **No Paid APIs Required:** Ingests public search endpoints for ■0 total API costs during POC.
- **Canonical Unit Normalizer:** Standardizes grams, kilograms, milliliters, and liter metrics.
- **Fuzzy Token Matcher:** Accurately pairs items across disparate catalog titles with >85% confidence score.
- **Dynamic Fallbacks:** Gracefully handles out-of-stock items and store closures.

Slide 8: Cost Analysis & Unit Economics — Zero Cost POC to Lean Production Scale

- **POC Cost:** ■0 (Android Studio + Local FastAPI + Device GPS + Free Public Web Endpoints).
- **Scale Cost:** ~■500 - ■1,500/month (Basic Cloud VPS on Render/DigitalOcean + Redis).
- **Publishing:** $25 one-time Google Play Console developer registration fee.
- **Monetization Potential:** Affiliate commission on redirected sales, sponsored brand placements, and premium subscription for automated price alerts.

Slide 9: Security, Compliance & Anti-Scraping — Building a Sustainable Aggregation Engine

- **Rate-Limiting Defense:** 15-minute geo-grid caching reduces upstream hits by up to 80%.
- **Privacy First:** User location used purely for transient dark store resolution; no personal tracking.
- **Affiliate Alignment:** Drives qualified buying traffic directly into Blinkit & Zepto apps, creating win-win value.

Slide 10: Execution Roadmap & Milestones — From Proof of Concept to App Store Launch

- **Milestone 1 (Week 1-2):** Backend aggregator & Blinkit + Zepto adapters completed and tested.
- **Milestone 2 (Week 3-4):** Kotlin Android app UI, Location Provider, and Deep Linking completed.
- **Milestone 3 (Week 5):** Closed beta testing with 50 live users across 5 pincodes.
- **Milestone 4 (Week 6+):** Add Swiggy Instamart, Basket Optimizer & Play Store Launch.